Getting Started with GitHub Copilot

Learning Objectives

By the end of this lesson, you should be able to:

- Explain what GitHub Copilot is and how it supports software development.
- Describe the role of AI in GitHub Copilot.
- Identify common development tasks where Copilot can assist.
- Register for and activate GitHub Copilot.
- Install Visual Studio Code.
- Install and configure GitHub Copilot in Visual Studio Code.
- Authenticate Visual Studio Code with your GitHub account.
- Verify that GitHub Copilot is working correctly.
- Use GitHub Copilot Chat to perform a basic test.

---

1. Introduction to GitHub Copilot

Modern software development involves much more than simply knowing the syntax of a programming language. Developers frequently need to:

- remember APIs and library functions;
- write repetitive or boilerplate code;
- create tests;
- debug existing code;
- refactor implementations;
- explore unfamiliar frameworks;
- search documentation for examples;
- compare different approaches to solving the same problem.

AI-assisted development tools are designed to reduce some of this overhead.

One of the most widely used tools in this category is **GitHub Copilot**.

GitHub Copilot acts as an **AI-assisted pair programmer**. It works alongside you while you develop software and provides suggestions based on the context of your current work.

Instead of constantly switching between your IDE, documentation, search engines, and other resources, you can ask Copilot questions or receive suggestions directly inside your development environment.

NOTE — AI Pair Programming

Traditional pair programming involves two developers working together, usually with one developer writing code while the other reviews, suggests improvements, and considers the broader problem.

GitHub Copilot applies a similar idea using AI. The AI assists the developer, but the human developer remains responsible for understanding, reviewing, testing, and approving the resulting code.

---

2. What Is GitHub Copilot?

GitHub Copilot is an **AI-powered programming assistant** designed to help developers write code faster and with less repetitive effort.

It can analyze the context surrounding the code you are currently working on and generate suggestions such as:

- partial lines of code;
- complete lines;
- code blocks;
- methods or functions;
- tests;
- explanations;
- alternative implementations.

For example, suppose you begin writing a function:

```
public boolean isPrime(int number) {
```

Based on the method name, parameter, programming language, and surrounding code, Copilot may suggest an implementation for determining whether number is prime.

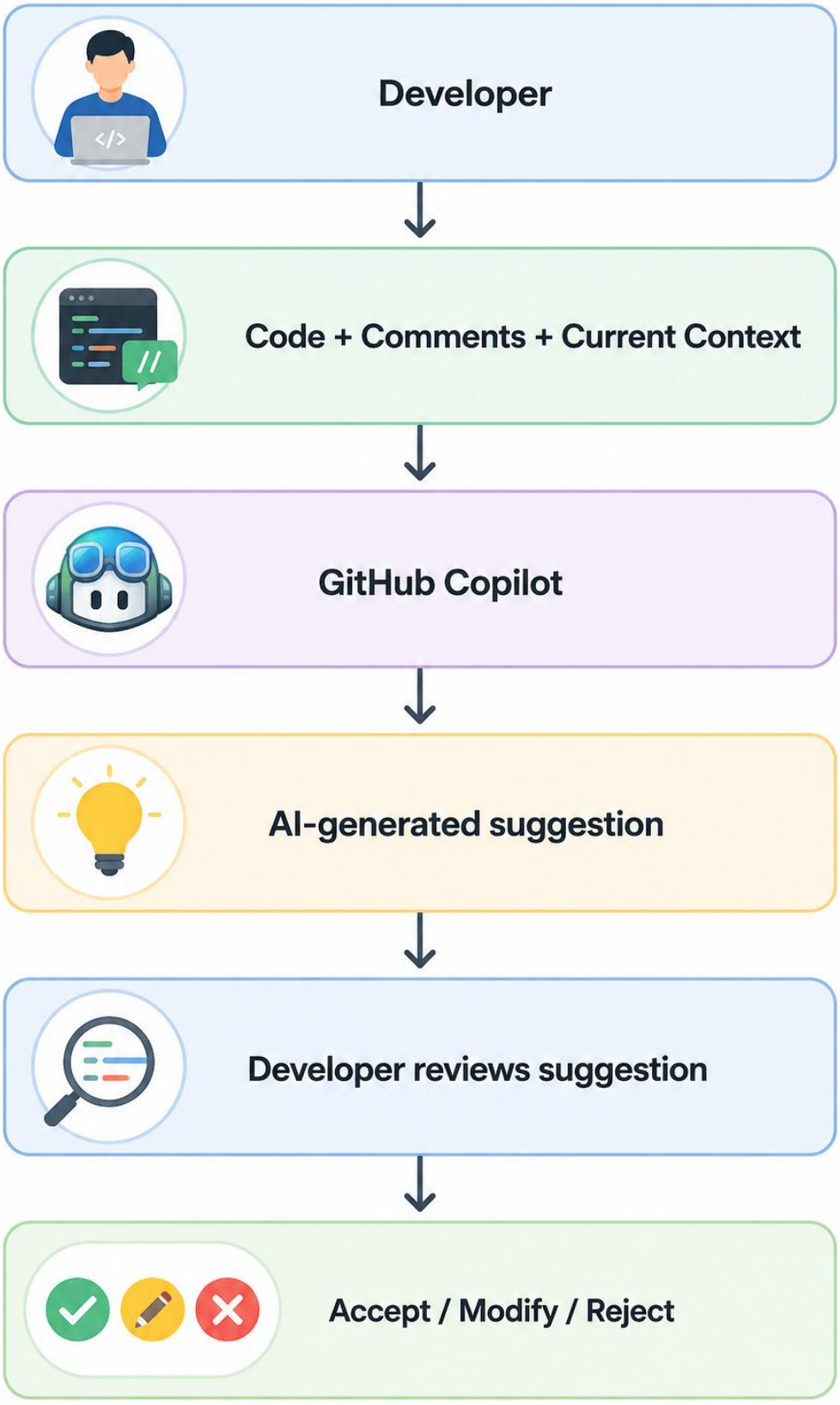
The important idea is **context**.

Copilot does not simply behave like a static code-snippet library. It attempts to understand what you are currently doing and generate a response that fits that context.

---

3. How GitHub Copilot Works

At a high level, the interaction can be represented as:


Developer
Code + Comments + Current Context
GitHub Copilot
AI-generated suggestion
Developer reviews suggestion
Accept / Modify / Reject

The developer remains an essential part of this workflow.

Copilot generates a **suggestion**, not a guaranteed correct solution.

For example, imagine that you write:

```
// Return the largest number in the array
```

and then begin:

```
public int findLargest(int[] numbers) {
```

Copilot may use:

1. the comment;
2. the method name;
3. the parameter type;
4. the programming language;
5. nearby code;

to predict an appropriate implementation.

TIP — Context Matters

Copilot generally produces better suggestions when your code clearly communicates its intention. Descriptive method names, meaningful variable names, useful comments, and well-structured code give the AI more useful context.

---

4. What Can GitHub Copilot Help With?

GitHub Copilot can assist with several stages of software development.

4.1 Code Completion

Copilot can suggest code while you type.

Instead of completing only keywords or variable names, it may generate an entire block of code.

For example:

```
public double calculateAverage(List<Integer> numbers) {
```

Copilot may suggest the body of the method based on the method signature.

4.2 Generating Functions and Methods

A developer can describe the desired behavior and allow Copilot to propose an implementation.

For example:

```
// Find all customers whose balance is greater than the specified amount
```

This description gives Copilot information about the expected operation.

This can be especially useful when writing:

- utility functions;
- collection-processing logic;
- validation functions;
- data transformations;
- REST-related code;
- repetitive CRUD operations.

4.3 Exploring Alternative Solutions

Programming problems often have multiple valid solutions.

Copilot can help you explore alternatives rather than immediately searching the web for another implementation.

For example, a collection might be processed using:

`for`

loops or using:

`Stream`

operations.

Examining alternative implementations can be useful for understanding trade-offs involving:

- readability;
- maintainability;
- performance;
- complexity;

- programming style.

4.4 Writing Tests

Testing often involves repetitive setup and similar test structures.

Copilot can help generate:

- unit-test templates;
- test cases;
- assertions;
- boundary cases;
- example input data.

For example, after implementing:

```
public int divide(int a, int b)
```

you might ask Copilot to suggest tests for:

- normal division;
- negative values;
- zero;
- division by zero;
- large values.

PITFALL — Generated Tests Are Not Automatically Complete

A generated test suite may miss important edge cases. Developers must still determine what behavior needs to be tested and whether the generated tests provide sufficient coverage.

4.5 Learning and Exploring APIs

Suppose you are working with an unfamiliar API.

Instead of repeatedly leaving the IDE to search for examples, you can ask Copilot questions about how an API might be used.

For example:

How do I create a REST endpoint in Spring Boot?

or:

How do I read a JSON file in Java?

Copilot can provide explanations and example code directly within the development environment.

This can make experimentation faster.

However, official documentation remains important, particularly when exact API behavior or version-specific details matter.

4.6 Debugging

Copilot can also assist when investigating code that does not behave as expected.

For example, you can provide a piece of code and ask:

Why can this code produce a **`NullPointerException`**?

Copilot may identify possible causes and suggest corrections.

4.7 Refactoring

Copilot can help suggest ways to improve existing code.

Typical refactoring tasks include:

- simplifying complex logic;
- extracting methods;
- improving variable names;
- removing duplication;
- reorganizing code;
- converting between programming styles.

WARNING — Copilot Is Not a Code Authority

Never assume that generated code is correct simply because it compiles. AI-generated code may contain logical errors, inefficient algorithms, incorrect API usage, security problems, or assumptions that do not match the requirements.

Treat Copilot output the same way you would treat code suggested by another developer: **review it, understand it, test it, and modify it when necessary.**

5. Copilot Augments Developers Rather Than Replacing Them

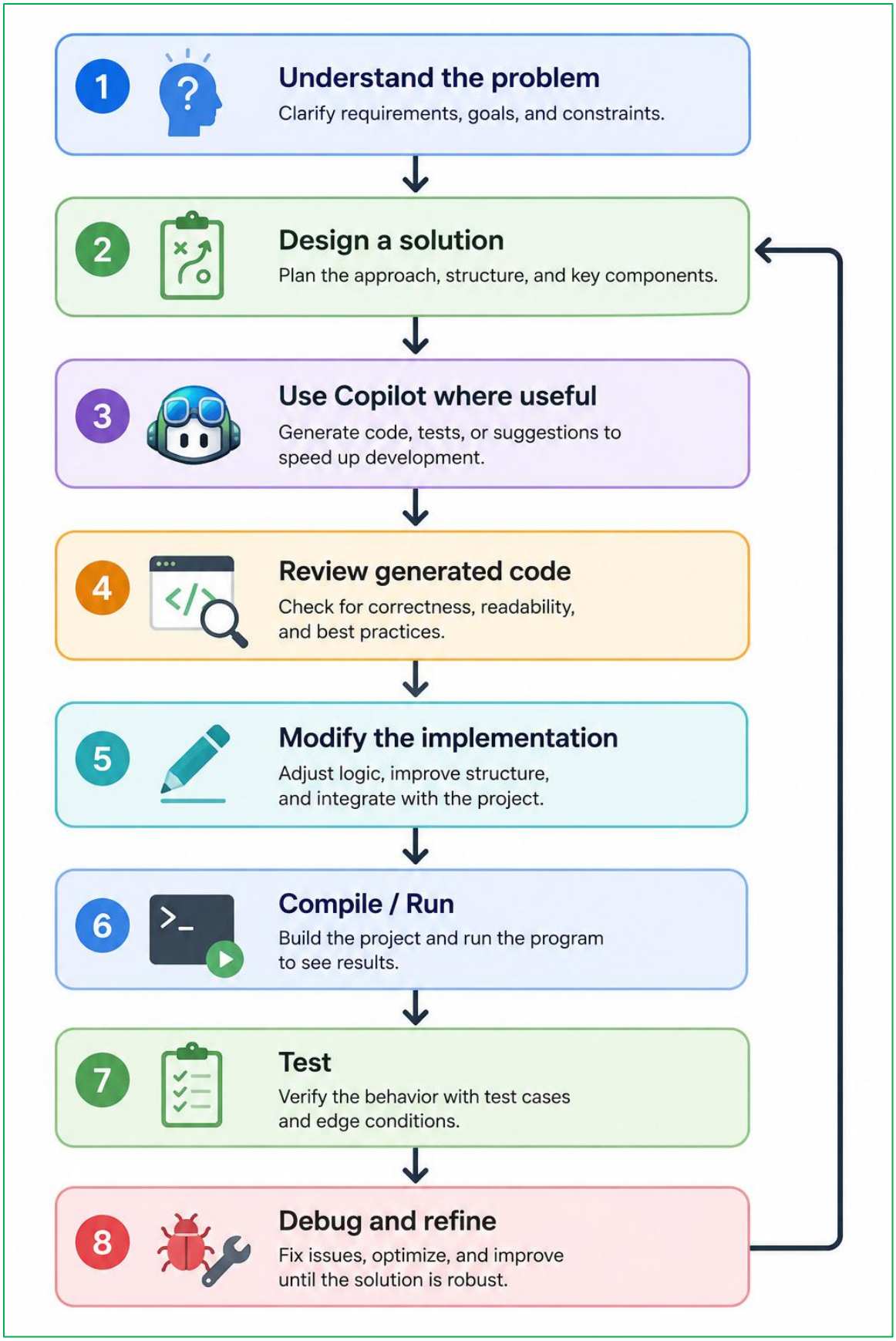
A central principle of AI-assisted development is:

The developer remains responsible for the software.

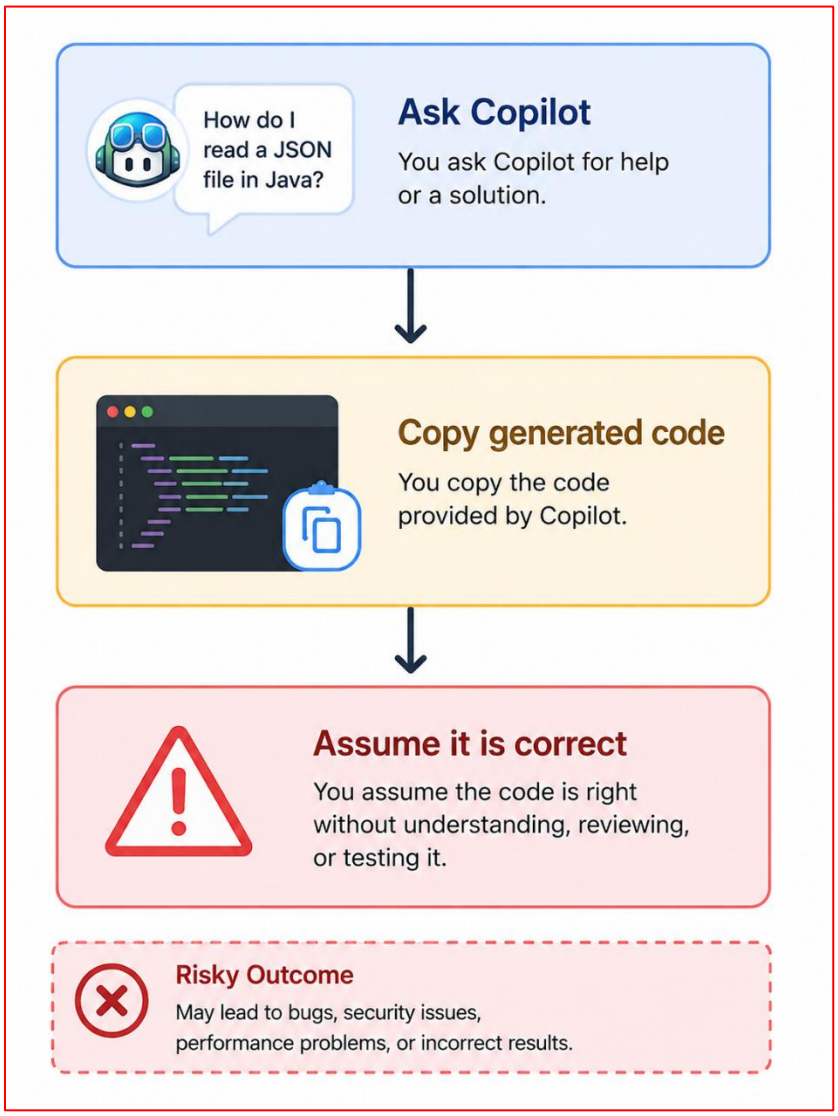
Copilot can reduce the amount of routine code you need to type, but it does not remove the need to understand:

- algorithms;
- data structures;
- object-oriented design;
- software architecture;
- databases;
- APIs;
- testing;
- debugging;
- security;
- performance;
- maintainability.

A productive workflow therefore looks like this:


1
Understand the problem
Clarify requirements, goals, and constraints.
2
Design a solution
Plan the approach, structure, and key components.
3
Use Copilot where useful
Generate code, tests, or suggestions to speed up development.
4
Review generated code
Check for correctness, readability, and best practices.
5
Modify the implementation
Adjust logic, improve structure, and integrate with the project.
6
Compile / Run
Build the project and run the program to see results.
7
Test
Verify the behavior with test cases and edge conditions.
8
Debug and refine
Fix issues, optimize, and improve until the solution is robust.

An unproductive workflow is:



TIP — Think First, Generate Second

Before asking Copilot to generate a solution, make sure you understand the problem you are trying to solve. AI assistance is significantly more useful when you can evaluate whether its proposed solution makes sense.

---

6. Preparing the Development Environment

We will now configure a development environment in which GitHub Copilot can be used.

The overall setup process is:

Step 1 → Obtain a GitHub account

Step 2 → Obtain GitHub Copilot access

Step 3 → Install Visual Studio Code

Step 4 → Install GitHub Copilot

Step 5 → Sign in to GitHub

Step 6 → Verify Copilot

Step 7 → Test Copilot Chat

---

7. Step 1 — Access GitHub Copilot

Open the GitHub Copilot page:

`github.com/features/copilot`

You can also search for **GitHub Copilot** using a search engine and open the official GitHub page.

Locate the option for getting started with Copilot.

Depending on GitHub's current interface, available plans and eligibility, you may be presented with different Copilot options.

Select the option appropriate for your GitHub account.

Complete the registration or activation process.

NOTE — Plans and Interfaces Change

GitHub may change Copilot plans, pricing, free-access options, trial availability, and the names or locations of buttons in its interface. Therefore, the exact screen you see may differ from screenshots or demonstrations used in class.

The important requirement is that the GitHub account you intend to use in Visual Studio Code has **GitHub Copilot access**.

---

8. Step 2 — Install Visual Studio Code

GitHub Copilot integrates with several development environments. In these notes, we will use **Visual Studio Code (VS Code)**.

Visual Studio Code is a free code editor developed by Microsoft. It is widely used because of its:

- extension ecosystem;
- support for many programming languages;
- integrated terminal;
- debugging tools;
- Git integration;
- project and file management;
- customization options.

Open:

code.visualstudio.com

Choose the installer corresponding to your operating system.

Common options include:

- Windows
- macOS
- Linux

Download and run the installer.

After installation, start **Visual Studio Code**.

PITFALL — Visual Studio vs. Visual Studio Code

Visual Studio and **Visual Studio Code** are different products.

For this tutorial, make sure you install **Visual Studio Code (VS Code)**.

---

9. Step 3 — Open the Extensions View

VS Code functionality can be extended using extensions.

GitHub Copilot is installed through this extension system.

In Visual Studio Code:

1. Locate the **Activity Bar** on the left side of the window.
2. Click the **Extensions** icon.

You can also use the keyboard shortcut:

Windows/Linux: Ctrl + Shift + X

macOS: Cmd + Shift + X

The Extensions view should now appear.

---

10. Step 4 — Search for GitHub Copilot

Inside the Extensions search field, enter:

GitHub Copilot

Locate the official GitHub Copilot extension.

Before installing an extension, verify that you are selecting the legitimate extension published by the expected publisher.

Click:

Install

VS Code will download and configure the extension.

Depending on the current version of the Copilot extension and VS Code, related Copilot chat functionality may also be installed or integrated.

TIP — Verify the Publisher

When installing development tools, do not rely only on an extension's name or icon. Check the publisher information as well. This is a useful general security practice when installing IDE extensions.

---

11. Step 5 — Verify the Installation

Return to the Extensions view and locate GitHub Copilot.

A successfully installed extension will no longer display the same **Install** action. Instead, VS Code will provide controls for managing or uninstalling the extension.

At this stage:

VS Code installed ✓

Copilot extension ✓

Authentication may still be required.

---

12. Step 6 — Sign In with Your GitHub Account

GitHub Copilot must know which GitHub account is using the service.

The GitHub account used inside Visual Studio Code should therefore be the account that has access to Copilot.

If VS Code is not already authorized, you may receive a sign-in prompt.

Select the option to:

Sign in to GitHub

Your browser may open automatically.

GitHub will display an authorization request for Visual Studio Code.

Review the request and, if appropriate, authorize Visual Studio Code.

After authorization, the browser should redirect you back to VS Code or indicate that authentication has completed successfully.

The authentication flow is approximately:

```
Visual Studio Code
        ↓
Sign in to GitHub
        ↓
Web browser opens
        ↓
GitHub authentication
        ↓
Authorization request
        ↓
Approve
        ↓
Return to VS Code
```

PITFALL — Wrong GitHub Account

Developers often have multiple GitHub accounts—for example, personal, school, and work accounts.

If Copilot appears unavailable after installation, check which GitHub account is currently authenticated in VS Code and confirm that the same account has Copilot access.

---

13. Step 7 — Check GitHub Copilot Status

After authentication, locate the Copilot controls or Copilot icon available in the VS Code interface.

Open the Copilot status information.

You should see an indication that Copilot is available or ready.

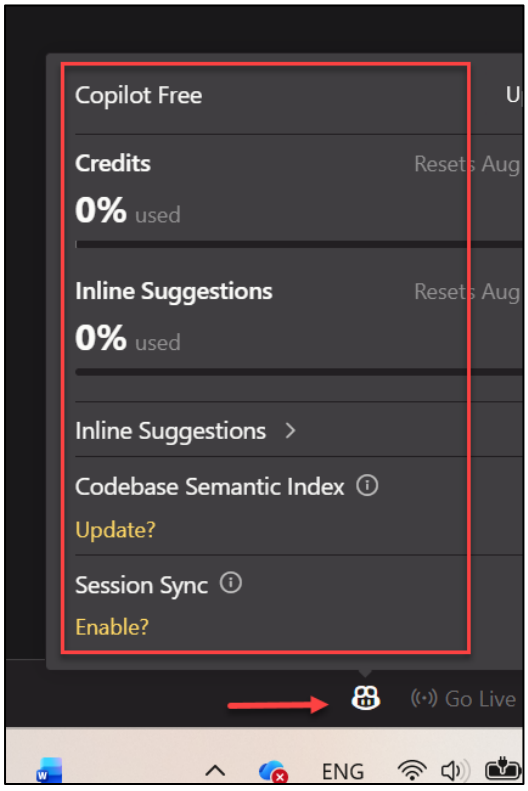
Conceptually, you want the environment to show:

GitHub account: Authenticated

Copilot: Ready

If Copilot is not ready, check:

1. Is the extension installed?
2. Are you signed into GitHub?
3. Are you using the correct GitHub account?
4. Does that account have Copilot access?
5. Is VS Code connected to the Internet?
6. Does VS Code need to be restarted or reloaded?



---

14. Step 8 — Open GitHub Copilot Chat

GitHub Copilot provides a conversational interface that allows you to ask programming-related questions.

Open the Copilot Chat interface from the available Copilot controls in VS Code.

You should see a chat area where prompts can be entered.

The interaction follows a simple model:

```
Your question
      ↓
Copilot Chat
      ↓
Generated explanation/code
```

```
      ↓
Your evaluation
```

---

15. Step 9 — Perform a Simple Test

Before using Copilot for programming tasks, perform a simple test to verify that the setup works.

In Copilot Chat, enter:

What is an API?

Submit the prompt.

Copilot should generate an explanation.

If you receive a response, you have verified several components at once:

- the extension is installed;
- GitHub authentication is working;
- your account has Copilot access;
- Copilot Chat can communicate with the service;
- VS Code integration is functioning.

TIP — Start with a Simple Prompt

When verifying a new development tool, begin with a simple request. This makes it easier to distinguish configuration problems from problems caused by a complex programming prompt.

---

16. Understanding Prompts

The instruction or question you give an AI system is commonly called a **prompt**.

For example:

What is an API?

is a simple explanatory prompt.

A programming-oriented prompt could be:

Write a Java method that receives an integer array and returns

the largest value.

A more detailed prompt might be:

Write a Java method called findLargest that receives an int array

and returns the largest value. Throw *IllegalArgumentException* if the array is null or empty.

Notice how the third prompt provides significantly more information.

It specifies:

- programming language;
- method name;
- input type;
- expected output;
- exceptional conditions.

As a general principle:

```
Clearer Context + Clearer Requirements
                  ↓
Potentially More Useful AI Output
```

This does **not** guarantee correctness, but it usually gives the AI a better description of the intended task.

---

17. Practical Exercise — Verify Your Copilot Environment

Complete the following exercise before proceeding to more advanced Copilot features.

Part A — Environment

- Open Visual Studio Code.
- Open the Extensions view.
- Search for GitHub Copilot.
- Confirm that the extension is installed.
- Confirm that you are signed into GitHub.
- Confirm that you are using the intended GitHub account.
- Confirm that Copilot reports that it is available or ready.

Part B — Chat Test

Open Copilot Chat and ask:

What is an API?

Read the response.

Then ask:

Explain REST API in simple terms.

Compare the two responses.

Part C — Programming Test

Enter:

Write a Java method that determines whether an integer is even.

Review the generated code.

Do **not** simply accept the result.

Answer these questions:

1. Does the generated method compile?
2. Is the method logically correct?
3. What happens with 0?
4. What happens with negative numbers?
5. Could you write the method yourself without Copilot?

The last question is particularly important.

The objective is not merely to obtain working code. The objective is to use AI while maintaining your own ability to understand and evaluate software.

---

18. Troubleshooting

Problem: Copilot Does Not Appear in VS Code

Check whether the extension is installed.

Open:

Extensions → GitHub Copilot

If necessary, install or enable it.

Problem: Copilot Requests Authentication

Sign into the GitHub account that has Copilot access and complete the authorization process.

Problem: Copilot Is Installed but Not Working

Check:

✓ Internet connection

✓ GitHub authentication

✓ Correct GitHub account

✓ Copilot access

✓ Extension enabled

✓ VS Code up to date

You may also try reloading or restarting Visual Studio Code.

Problem: Copilot Generates Incorrect Code

This does not necessarily indicate a configuration problem.

AI-generated code can be incorrect.

Use the standard development process:

Read → Understand → Compile → Test → Debug → Improve

WARNING — Never Skip Code Review

AI-generated code must be treated as untrusted code until you have inspected and tested it. This becomes especially important when working with authentication, authorization, databases, file operations, external input, concurrency, or other security-sensitive functionality.

---

19. Key Concepts

The main concepts introduced in this lesson are:

Concept	Description
GitHub Copilot	AI-assisted programming tool for developers
AI Pair Programming	Using AI as an assistant during software development
Context	Information Copilot uses to generate relevant suggestions
Code Completion	AI-generated continuation of code being written
Copilot Chat	Conversational interface for asking development questions
Prompt	Instruction or question supplied to the AI
VS Code Extension	Component that integrates Copilot into Visual Studio Code
Authentication	Connecting VS Code to an authorized GitHub account
Human Review	Developer verification of generated code before use

---

20. Recommended Copilot Mindset

GitHub Copilot should be viewed as a **development assistant**, not as an automatic solution generator.

Use it to:

```
Generate ideas
      ↓
Explore implementations
      ↓
Reduce repetitive work
      ↓
Learn unfamiliar APIs
      ↓
Generate initial code
      ↓
Assist testing/debugging
```

But always follow this with:

```
Understand
    ↓
Review
    ↓
Verify
```

```
    ↓
Test
    ↓
Improve
```

NOTE — The Developer Is Still Responsible

Copilot can generate code quickly. It cannot transfer responsibility for that code away from the developer.

If you include AI-generated code in a project, you are still responsible for understanding its behavior and ensuring that it satisfies the project's functional, quality, security, and maintainability requirements.

---

21. Lesson Summary

GitHub Copilot is an AI-powered programming assistant integrated into development environments such as Visual Studio Code.

It can assist developers with:

- code completion;
- method and function generation;
- alternative implementations;
- testing;
- debugging;
- refactoring;
- API exploration;
- programming explanations.

In this lesson, we also configured the basic Copilot development environment:

```
GitHub account
      ↓
GitHub Copilot access
      ↓
Visual Studio Code
      ↓
GitHub Copilot extension
      ↓
GitHub authentication
      ↓
Copilot ready
      ↓
```

`Copilot Chat test`

The most important principle to remember is that **Copilot assists the developer; it does not replace the developer's judgment**.

Generated code should always be understood, reviewed, tested, and validated before being incorporated into a software project.